

Control de flujo

Cómo evitar que un emisor rápido desborde a un receptor lento

MODELO OSI

Application
Transport
Network
Link
Physical

Metas del capítulo

Lo que ya hemos visto y hacia dónde vamos en este tema

YA VISTO ✓

01

Entrega de datos confiable

02

Protocolo de parada y espera

03

Protocolos de tubería

EN ESTE CAPÍTULO →

04

Control de flujo en general

Soluciones generales de reserva de búferes

05

Control de flujo en TCP

Ventana deslizante aplicada al protocolo real

La meta del control de flujo: *que el emisor nunca envíe más datos de los que el receptor puede procesar.*

Control de flujo

DEFINICIÓN

Evitar que un emisor rápido desborde a un receptor lento

El emisor debe ajustar su ritmo de envío a la capacidad de procesamiento de datos del **receptor**, evitando así la pérdida de información por **desbordamiento de búferes**.

CONTEXTO — ¿EN QUÉ CAPA?

Capa de enlace de datos

Controla flujo entre **dos máquinas vecinas** conectadas directamente (host↔router o router↔router).

Capa de transporte ← este capítulo

Controla flujo **extremo a extremo** entre procesos en los hosts, sin importar cuántos saltos intermedios haya.

En este capítulo veremos cómo funciona el control de flujo en la capa de transporte.

¿Por qué el receptor necesita búferes?

Premisa: el receptor mantiene búferes para **almacenar los segmentos** que llegan **antes de procesarlos y enviarlos a la aplicación** en la capa superior (p. ej., `socket recv`).

ESTO ES NECESARIO PORQUE...

01

El emisor puede ir más rápido

Si los **segmentos llegan más rápido** de lo que el receptor puede procesarlos, hay que guardarlos temporalmente en algún sitio.

02

Procesar en lotes es más eficiente

Acumular varios segmentos antes de pasarlos a la capa de aplicación reduce la sobrecarga de procesamiento.

03

Los segmentos llegan desordenados

Si llegan segmentos posteriores antes que los previos, hay que **almacenarlos hasta completar el orden correcto** (p. ej., el sistema de retransmisión selectiva usado en **TCP**).

→ Sin búferes, cualquier desajuste de velocidad o desorden causa pérdida de datos.

Problema 1: la aplicación lee lenta

CAUSA DESDE LA CAPA DE APLICACIÓN

Situación

La aplicación receptora no lee los datos al instante: está ocupada con otras tareas y tarda en retirar los segmentos del búfer.

QUÉ PASA EN EL RECEPTOR



Consecuencia

Si la aplicación tarda demasiado en leer, el emisor **satura los búferes del receptor** y se pierden (*drop*) datos.

Problema 2: la red causa retrasos

CAUSA DESDE LA CAPA DE RED

La capa de red **también** puede dejar al receptor con menos capacidad de búfer.

SITUACIÓN 1

Un paquete llega dañado

Un enrutador en la ruta corrompe un paquete. El receptor lo detecta al llegar.

→ Los paquetes buenos que llegan después **no pueden entregarse a la aplicación hasta que se retransmita el dañado**. Mientras, ocupan búfer.

SITUACIÓN 2

Cambio de rutas

El algoritmo de enrutamiento cambia rutas lentas por rutas más rápidas.

→ Algunos paquetes **llegan fuera de orden** y hay que almacenarlos hasta recomponer la secuencia y entregarlos a la aplicación.

En ambos casos: *el búfer del receptor se ve obligado a retener paquetes durante más tiempo del previsto.*

Problema 3: muchas conexiones a la vez

CAUSA DESDE LA PROPIA CAPA DE TRANSPORTE

La capa de transporte comparte un **presupuesto fijo de memoria** entre todas las conexiones abiertas del host.

QUÉ OCURRE CUANDO AUMENTAN LAS CONEXIONES

ANTES · pocas conexiones



Cada conexión recibe un búfer grande.

Procesamiento fluido, sin pérdidas.

DESPUÉS · muchas conexiones



El búfer por conexión se reduce.

Aumenta el riesgo de desbordamiento.

→ Sumado a las situaciones de la aplicación y la red, esto puede producir **desbordamiento de búferes** en el receptor. (Esto es particularmente desafiante en dispositivos móviles o tipo IoT.)

¿No lo hace ya la capa de enlace?

Pregunta clave

Si la capa de enlace ya hace control de flujo, ¿por qué lo necesita también la capa de transporte?

¿No lo hace ya la capa de enlace?

Pregunta clave

Si la capa de enlace ya hace control de flujo, ¿por qué lo necesita también la capa de transporte?

CAPA DE ENLACE

Solo ve el salto vecino

Controla flujo entre dos nodos físicamente conectados.

No sabe:

- si la aplicación lee lenta
- si hay muchas conexiones abiertas
- si hubo reordenamiento de paquetes

CAPA DE TRANSPORTE

Ve todo el camino extremo a extremo

Es la única que puede reaccionar a los problemas vistos en las slides anteriores.

Necesita su propio protocolo para:

- ajustar el ritmo según el receptor real
- evitar que se desborden sus búferes

Conclusión

Los protocolos de capa de enlace confiables (ack, retransmisión) **no bastan** para evitar el desbordamiento de búferes. Hace falta un protocolo específico de **control de flujo en la capa de transporte**.

Estrategias de reparto de búferes

Problema

Si el receptor tiene varias conexiones abiertas, ¿cómo reparte los búferes entre ellas?

DOS ENFOQUES POSIBLES

ENFOQUE A

Pool compartido

Todas las conexiones comparten el mismo conjunto de búferes.

✓ Aprovecha mejor la memoria si el tráfico es desigual.

✗ Una conexión ruidosa puede afectar a las demás.

ENFOQUE B

Búferes dedicados

Cada conexión tiene un conjunto de búferes específico y reservado.

✓ Aísla a cada conexión de las demás.

✗ Puede desperdiciar memoria si unas conexiones no la usan.

→ Veremos una estrategia dinámica que mezcla ambas ideas.

Primer intento: aceptar si hay búfer libre

Problema

¿Qué hace el receptor cuando le llega un nuevo segmento?

PRIMERA SOLUCIÓN (INGENUA)

1. Llega un segmento



2. ¿Hay búfer libre?



SÍ ✓

Se acepta el segmento

NO ✗

Se descarta el segmento

¿SIRVE ESTO COMO CONTROL DE FLUJO?

⚠ NO · limitaciones

El emisor sigue enviando a ciegas.

No sabe cuánto espacio tiene el receptor, así que cuando los búferes se llenan, los segmentos se pierden.

→ *Retransmisiones innecesarias, latencia extra, tráfico desperdiciado.*

→ Necesitamos un **protocolo** donde el emisor **sepa de antemano** cuánto espacio tiene el receptor.

Hacia un protocolo dinámico

LO QUE YA SABEMOS

- ✓ Los búferes del receptor se pueden llenar por distintas causas.
- ✓ El receptor y el emisor deben ajustar **dinámicamente** sus asignaciones.
- ✓ Esto significa **ventanas de tamaño variable**.

EL PROBLEMA: EMISOR Y RECEPTOR SABEN COSAS DISTINTAS

El emisor sabe...

cuántos datos le gustaría enviar

pero no cuántos puede enviar realmente.

El receptor sabe...

cuánto espacio libre tiene

pero no cuándo el emisor querrá enviar.

Pregunta central

¿Qué reglas debería cumplir un protocolo que permita a ambos **negociar ese espacio dinámicamente**?

La idea: pedir y otorgar búferes

Una Posible Solución

El emisor **solicita** espacio de búfer al receptor, y el receptor le **otorga** explícitamente cuánto puede usar.

CÓMO PIENSA CADA LADO

EMISOR

Solicita espacio

“Sé cuántos datos quiero enviar — pido N búferes.”

Así evita:

- enviar más de lo que el receptor aguanta
- saturar búferes y perder datos

RECEPTOR

Otorga lo que puede

“Según mi carga, reservo X búferes para esta conexión.”

Debe poder:

- repartir búferes entre varias conexiones
- ajustar dinámicamente si aumenta la carga

→ Esta negociación funciona como un **contrato dinámico**: el emisor no asume nada, siempre pregunta.

El protocolo, paso a paso (1/2)

Funcionamiento de la comunicación emisor ↔ receptor usando la solución anterior.

01

El emisor solicita búferes

Basándose en lo que necesita enviar, pide una cantidad inicial al receptor.

02

El receptor responde con un "grant"

Otorga tantos búferes como puede, según su carga actual. Mensaje típico: "te reservo X búferes".

03

Empieza el intercambio

El emisor ya sabe cuánto puede enviar. A partir de aquí, el receptor seguirá actualizando la reserva.

OPTIMIZACIÓN: PIGGYBACKING

¿Enviar ACKs y grants por separado?

No hace falta: el **receptor** puede **incorporar la reserva de búferes y el ACK en un mismo segmento**. Así se ahorran mensajes.

Continúa en la siguiente slide →

El protocolo, paso a paso (2/2)

Cómo contabiliza el emisor su asignación de búferes.

EL EMISOR LLEVA UN CONTADOR

Asignación inicial: 5 (otorgada por el receptor)

Envía 1 segmento → asignación = 4

Envía 1 segmento → asignación = 3 → ... → asignación = 0

REGLA CRÍTICA

Si la asignación disponible llega a 0,
el emisor **debe detenerse por completo** hasta que el receptor le otorgue más búferes.

Intuición: el emisor tiene un **crédito** que gasta con cada envío y el receptor lo va recargando.

Comunicación emisor ↔ receptor

PARÁMETROS

Nº de secuencia

4 bits

rango: 0 ... 15

Búferes en el receptor

4 en total

todos del mismo tamaño

Estrategia aplicada

El protocolo básico

pedir y otorgar búferes

EJERCICIO: Lo Hacemos Paso a Paso

Exploración:

Y vamos a descubrir un problema: **deadlock**

QUÉ MOSTRAR EN LA SOLUCIÓN

· Nº de secuencia por segmento · Búferes otorgados y ACKs en segmentos de respuesta · Asignación nº de secuencia → búferes

Resolución — paso a paso

El emisor pide 8
búferes.



Resolución — paso a paso

El emisor pide 8 búferes.

	<u>A</u>	<u>Message</u>
1	→	< request 8 buffers >

<u>B</u>	<u>B's Buffer</u>	<u>Comments</u>
→	_____	A wants 8 buffers

El receptor otorga 4 búferes y espera el segmento de Nº secuencia 0.

	<u>A</u>	<u>Message</u>
1	→	< request 8 buffers >
2	←	<ack = 15, buf = 4>

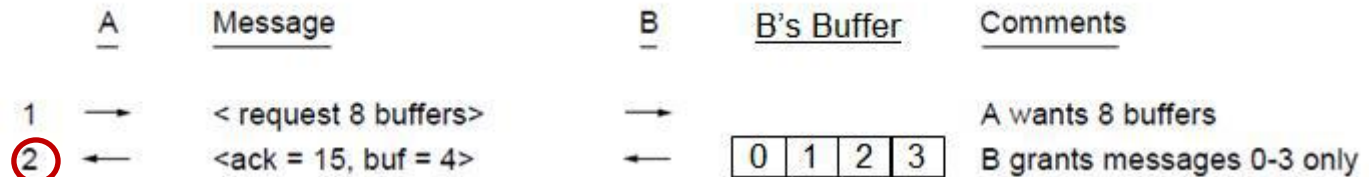
<u>B</u>	<u>B's Buffer</u>	<u>Comments</u>
→		A wants 8 buffers
←	0 1 2 3	B grants messages 0-3 only

Resolución — paso a paso

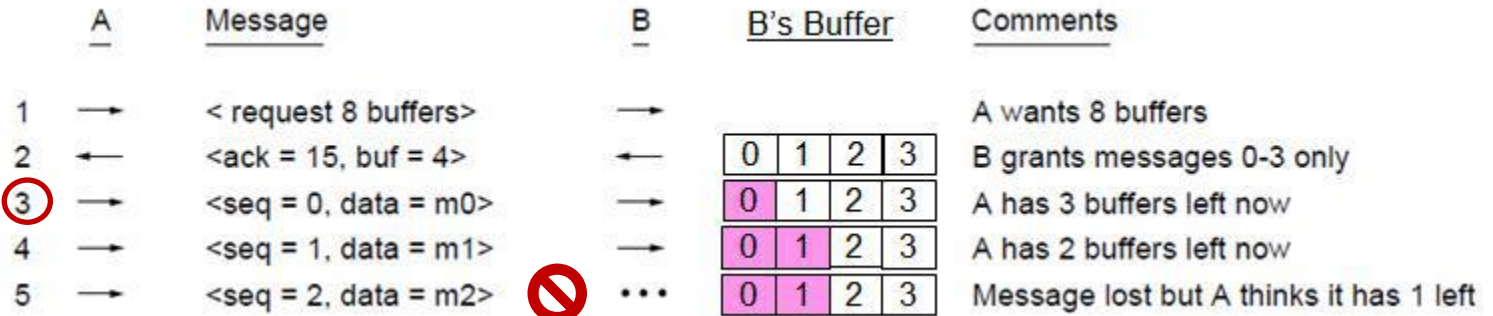
El emisor pide 8 búferes.



El receptor otorga 4 búferes y espera el segmento de Nº secuencia 0.



El emisor envía 3 segmentos; los 2 primeros llegan y el 3º se pierde.



Resolución — paso a paso

El receptor confirma los 2 primeros segmentos y otorga 3 búferes.

A	Message	B	B's Buffer	Comments
1 →	< request 8 buffers>	→		A wants 8 buffers
2 ←	<ack = 15, buf = 4>	←	0 1 2 3	B grants messages 0-3 only
3 →	<seq = 0, data = m0>	→	0 1 2 3	A has 3 buffers left now
4 →	<seq = 1, data = m1>	→	0 1 2 3	A has 2 buffers left now
5 →	<seq = 2, data = m2>	...	0 1 2 3	Message lost but A thinks it has 1 left
6 ←	<ack = 1, buf = 3>	←	1 2 3 4	B acknowledges 0 and 1, permits 2-4

El emisor envía 2 segmentos nuevos (llegan) y reenvía el segmento perdido.

A	Message	B	B's Buffer	Comments
1 →	< request 8 buffers>	→		A wants 8 buffers
2 ←	<ack = 15, buf = 4>	←	0 1 2 3	B grants messages 0-3 only
3 →	<seq = 0, data = m0>	→	0 1 2 3	A has 3 buffers left now
4 →	<seq = 1, data = m1>	→	0 1 2 3	A has 2 buffers left now
5 →	<seq = 2, data = m2>	...	0 1 2 3	Message lost but A thinks it has 1 left
6 ←	<ack = 1, buf = 3>	←	1 2 3 4	B acknowledges 0 and 1, permits 2-4
7 →	<seq = 3, data = m3>	→	1 2 3 4	A has 1 buffer left
8 →	<seq = 4, data = m4>	→	1 2 3 4	A has 0 buffers left, and must stop
9 →	<seq = 2, data = m2>	→	1 2 3 4	A times out and retransmits

Resolución — paso a paso

El receptor confirma todos los segmentos y otorga 0 búferes.

El receptor otorga 1 búfer, luego 2.

El emisor manda 2 segmentos de datos.

A	Message	B	B's Buffer	Comments
1 →	< request 8 buffers>	→		A wants 8 buffers
2 ←	<ack = 15, buf = 4>	←	0 1 2 3	B grants messages 0-3 only
3 →	<seq = 0, data = m0>	→	0 1 2 3	A has 3 buffers left now
4 →	<seq = 1, data = m1>	→	0 1 2 3	A has 2 buffers left now
5 →	<seq = 2, data = m2>	...	0 1 2 3	Message lost but A thinks it has 1 left
6 ←	<ack = 1, buf = 3>	←	1 2 3 4	B acknowledges 0 and 1, permits 2-4
7 →	<seq = 3, data = m3>	→	1 2 3 4	A has 1 buffer left
8 →	<seq = 4, data = m4>	→	1 2 3 4	A has 0 buffers left, and must stop
9 →	<seq = 2, data = m2>	→	1 2 3 4	A times out and retransmits
10 ←	<ack = 4, buf = 0>	←	1 2 3 4	Everything acknowledged, but A still blocked
11 ←	<ack = 4, buf = 1>	←	2 3 4 5	A may now send 5
12 ←	<ack = 4, buf = 2>	←	3 4 5 6	B found a new buffer somewhere
13 →	<seq = 5, data = m5>	→	3 4 5 6	A has 1 buffer left
14 →	<seq = 6, data = m6>	→	3 4 5 6	A is now blocked again
15 ←	<ack = 6, buf = 0>	←	3 4 5 6	A is still blocked
16 ...	<ack = 6, buf = 4>	←	7 8 9 10	Potential deadlock

¡El emisor envía 4 búferes, pero se pierde el mensaje!
Deadlock!

El riesgo: deadlock

GENERALIZANDO EL ÚLTIMO PASO DEL EJERCICIO...

Escenario peligroso

Un segmento de reserva de búferes (sin datos) se pierde.

El emisor sigue esperando permiso; el receptor cree haberlo otorgado.

CONSECUENCIA

Emisor

Espera un grant que nunca llegará.

Receptor

Cree que el emisor ya tiene permiso.

→ Los dos se quedan esperando indefinidamente · DEADLOCK

SOLUCIÓN

Segmento de control periódico

Cada host envía cada cierto tiempo un segmento con el **ACK acumulado y el estado actual de sus búferes**.

→ Aunque un mensaje se pierda, el siguiente segmento periódico restaurará el estado y el estancamiento se romperá tarde o temprano.

Pero esto era sólo un ejemplo de un posible protocolo simple para entender el problema general del control de flujo...

PARTE 2 DEL CAPÍTULO

Ahora: control de flujo en TCP

Aplicamos las mismas ideas (control dinámico, reserva de búferes) al protocolo real más usado.

TCP da libertad de implementación

TCP no obliga a implementaciones estrictas. En particular...

NO SE REQUIERE QUE...

01

El emisor envíe al instante

No hace falta mandar datos en cuanto llegan de la aplicación.

02

El receptor confirme al instante

Los ACK pueden agruparse y retrasarse ligeramente.

03

El receptor entregue al instante

Los datos pueden quedarse en el búfer hasta que convenga entregarlos.

¿Por qué importa?

Esta **flexibilidad** se puede **explotar para mejorar el rendimiento**:

agrupar segmentos pequeños, retrasar ACKs para aprovechar **piggybacking**, sincronizar lecturas con el ciclo de la CPU, etc.

TCP requiere otro enfoque

Problema

No se puede usar el protocolo de búferes anterior tal cual en TCP.

¿POR QUÉ?

ANTES · protocolo genérico

Nº de secuencia = nº de paquete

Cada búfer ocupado tenía un número de paquete asociado.

Por eso el receptor podía decir: “te he reservado X búferes”.

EN TCP

Nº de secuencia = posición de byte

Los números de secuencia representan bytes dentro del flujo, no paquetes individuales.

El receptor sólo puede decir: “tengo estos rangos de bytes en búfer”.

→ Necesitamos un mecanismo distinto para anunciar espacio.

Dos simplificaciones que TCP permite

Dado que TCP identifica bytes del flujo (no paquetes), se pueden hacer dos mejoras:

MEJORA 1

No se guardan las cabeceras

En el esquema anterior, el receptor almacenaba paquetes completos — con cabeceras incluidas, que ocupan espacio.

En TCP:

- solo se guarda el **payload útil** del flujo
- el búfer retiene **más datos útiles** en el mismo espacio

MEJORA 2

El emisor ya no pide búferes

No necesita enviar un mensaje “pido N búferes” como en el protocolo anterior.

En TCP:

- el **receptor anuncia por iniciativa propia** cuánto espacio tiene
- se elimina el primer paso del handshake para solicitar búferes

El búfer circular del receptor

Cada conexión TCP tiene, en el receptor, un **búfer circular** de tamaño **RcvBuffer**.

VISUALIZACIÓN



Cuando la app lee datos...

...los retira del inicio del búfer y **libera espacio** para datos nuevos al final.

Esto es exactamente el socket `recv`!

```
import socket
data = s.recv(1024) # read 1024 bytes
```

Ventaja del circular

No importa dónde esté cada byte; lo relevante es **cuánto espacio libre queda**.

→ Ese espacio libre es lo que el receptor anunciará al emisor (siguiente slide).

La ventana: cuánto puede enviar el emisor

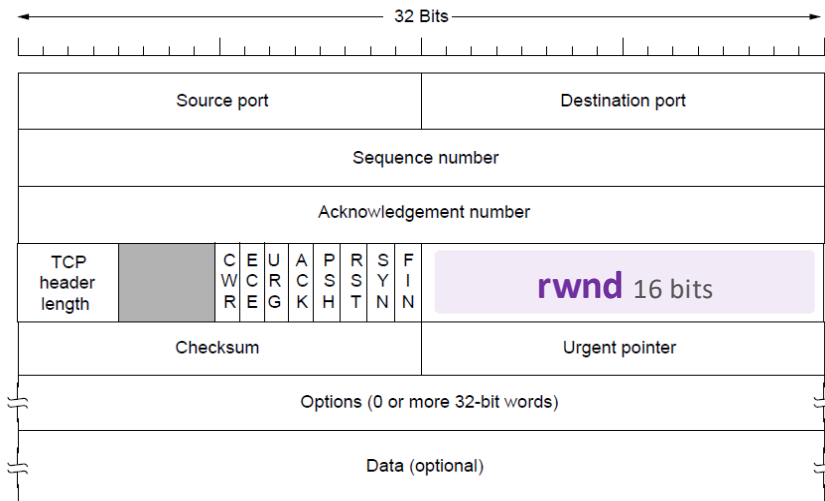
Pregunta

¿Cómo le dice el receptor al emisor cuánto espacio libre tiene?

Respuesta: la ventana de recepción

En cada segmento que envía (incluido el ACK), el **receptor** incluye un campo llamado **rwnd** (*receive window*) con el número de **bytes consecutivos que puede aceptar a partir de ahora**.

¿DÓNDE VIAJA ESE VALOR?



Formato TCP

Regla para el emisor

Nunca puede tener en vuelo (enviados y sin confirmar) más de **rwnd** bytes.

Si los alcanza, debe esperar a que el receptor anuncie más espacio.

El búfer y la ventana, visualmente

ESTADO DEL BÚFER DEL RECEPTOR

Tres regiones

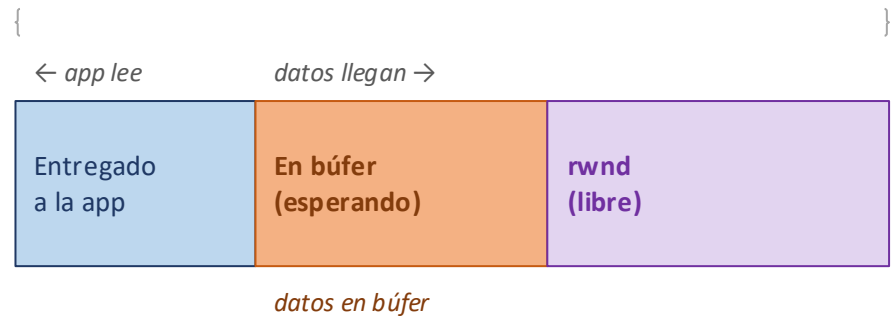
1. Datos ya entregados a la app
2. Datos recibidos, esperando entrega
3. Espacio libre = **rwnd**

Fórmula

$$\text{rwnd} = \text{RcvBuffer} - \text{datos en búfer}$$

Es decir: cuánto espacio queda libre en este instante.

RcvBuffer (tamaño total)



$$\text{rwnd} = \text{RcvBuffer} - \text{datos en búfer}$$

Esquema del búfer del receptor y su espacio libre (rwnd)

El emisor también tiene un búfer

El emisor también almacena los datos que envía en un búfer circular propio.

Los mantiene ahí hasta recibir el ACK, por si tiene que retransmitir.

¿CUÁNTO PUEDE ENVIAR REALMENTE?

bytes enviables = min (tamaño del búfer del emisor, rwnd)

El emisor nunca puede enviar más bytes que el menor de estos dos límites.

LÍMITE PROPIO

Su búfer

No tiene sentido enviar más de lo que cabe en su propia memoria esperando confirmación.

LÍMITE DEL RECEPTOR

El rwnd anunciado

Si envía más, el receptor no podrá aceptarlo y los bytes extra se descartarán.

Fórmula de rwnd: cómo la calcula el receptor

Fórmula

$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$$

tamaño del búfer – lo que está esperando ser leído

QUÉ SIGNIFICA CADA VARIABLE

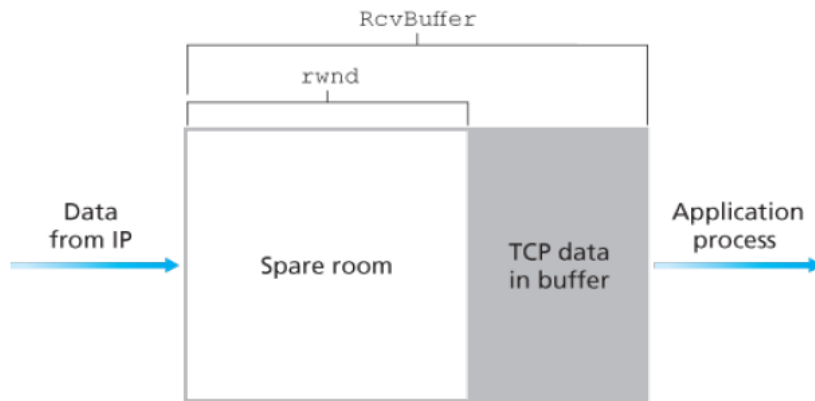


Figure 3.38 The receive window (*rwnd*) and the receive buffer (*RcvBuffer*)

rwnd *receive window*

espacio libre en el búfer que el receptor anuncia.

RcvBuffer

tamaño total del búfer asignado.

LastByteRcvd

último byte llegado al receptor.

LastByteRead

último byte leído por la aplicación.

Reglas del receptor en TCP

Qué hace el receptor en cada momento del control de flujo TCP.

01

Cuando llegan bytes en orden y en secuencia, los coloca en el búfer de recepción.

02

Al confirmar (ACK) la llegada de datos, anuncia al emisor el nuevo tamaño de ventana (rwnd).

03

Si el búfer está lleno, anuncia $\text{rwnd} = 0$ → el emisor debe pausar los envíos.

04

Cuando la aplicación lee X bytes del búfer, libera ese espacio y puede anunciar $\text{rwnd} = X$.

Intuición: *el receptor avisa continuamente al emisor cuánto puede aceptar.*

Reglas del emisor en TCP

Qué reglas debe respetar el emisor usando el `rwnd` que le anuncia el receptor.

REGLA 1 · VENTANA CERO

Si el receptor anuncia `rwnd = 0`, el emisor **no puede enviar datos**.

REGLA 2 · LÍMITE DE DATOS EN VUELO

Los bytes enviados pero aún no confirmados nunca pueden exceder la ventana:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

Intuición

El emisor puede tener **como máximo `rwnd` bytes en tránsito**. Cada vez que recibe un ACK, libera crédito y puede enviar más datos.

Ejercicio Paso a Paso

Situación: Una conexión emisor → receptor. El receptor tiene un **búfer circular de 4 KB**.

SECUENCIA DE EVENTOS EN EL BÚFER

1. **Búfer vacío**
2. **Búfer con 2 KB**
3. **Búfer con 4 KB (lleno)**
4. **App lee 2 KB**
5. **Búfer con 3 KB**

QUÉ HAY QUE MOSTRAR

- ▶ Tamaños y números de secuencia de los segmentos enviados.
- ▶ Tamaño de ventana y ACK en los segmentos recibidos.
- ▶ Cómo varía el uso del búfer circular.



Vamos a resolverlo paso a paso →

Emisor envía, receptor recibe

PASO 1

Búfer vacío

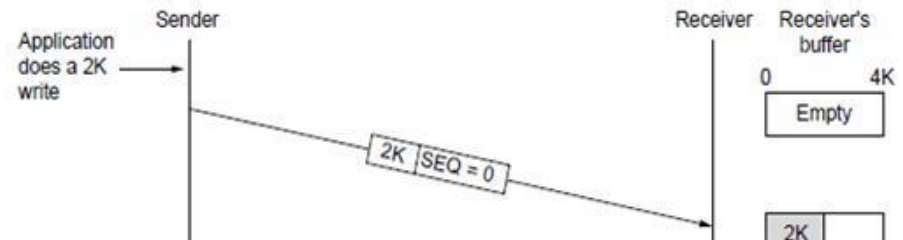
El receptor anuncia la ventana completa (4 KB).

PASO 2

Búfer con 2 KB

Llega el primer segmento de 2 KB. El receptor lo coloca en el búfer y reenvía ACK + WIN reducida.

DIAGRAMA PARCIAL



LO QUE MUESTRA EL DIAGRAMA

La ventana anunciada (WIN) empieza en 4 KB y se reduce a 2 KB tras recibir el primer segmento. El emisor solo puede enviar hasta ese límite; el receptor lo re-publica en cada ACK.

Continúa con el búfer lleno →

El búfer del receptor se llena

PASO 3

Búfer con 4 KB (lleno)

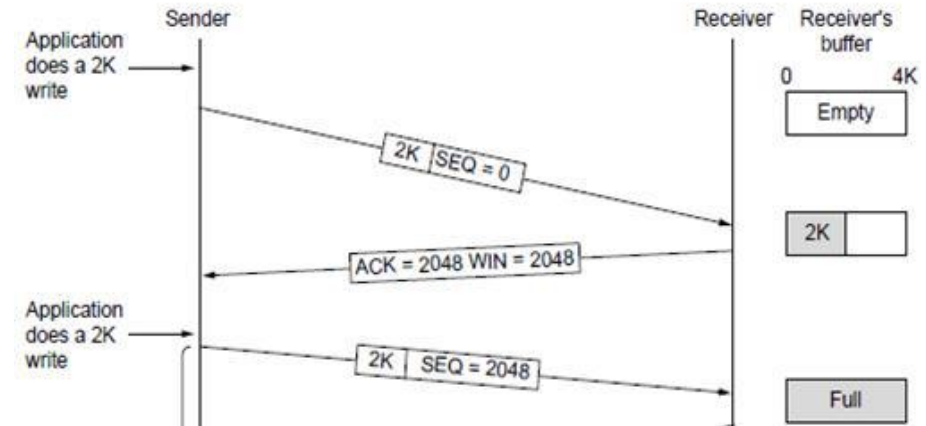
Llega un nuevo segmento de 2 KB que completa el búfer.

El receptor responde con:

$ACK + WIN = 0$

Significa: el emisor no puede enviar más datos hasta nuevo aviso.

DIAGRAMA PARCIAL — PASO 3



LO QUE MUESTRA EL DIAGRAMA

Al llegar el tercer segmento el búfer se llena y el receptor responde $ACK + WIN = 0$. A partir de aquí el emisor queda bloqueado hasta que la app consuma datos.

Cuando la aplicación lea, se libera espacio →

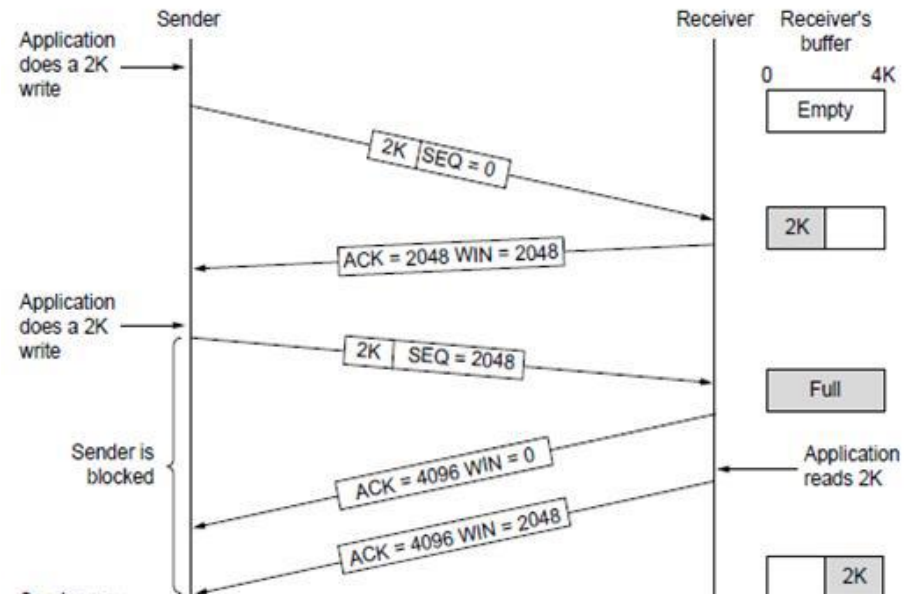
La app lee → el emisor reanuda

PASO 4

La app lee 2 KB

Se liberan 2 KB del búfer. El receptor anuncia **WIN = 2 KB**.

DIAGRAMA PARCIAL — PASOS 4-5



LO QUE MUESTRA EL DIAGRAMA

La app lee 2 KB → el receptor anuncia WIN = 2 KB → el emisor recupera crédito y reanuda el envío.

Resolución completa en la siguiente slide →

La app lee → el emisor reanuda

PASO 4

La app lee 2 KB

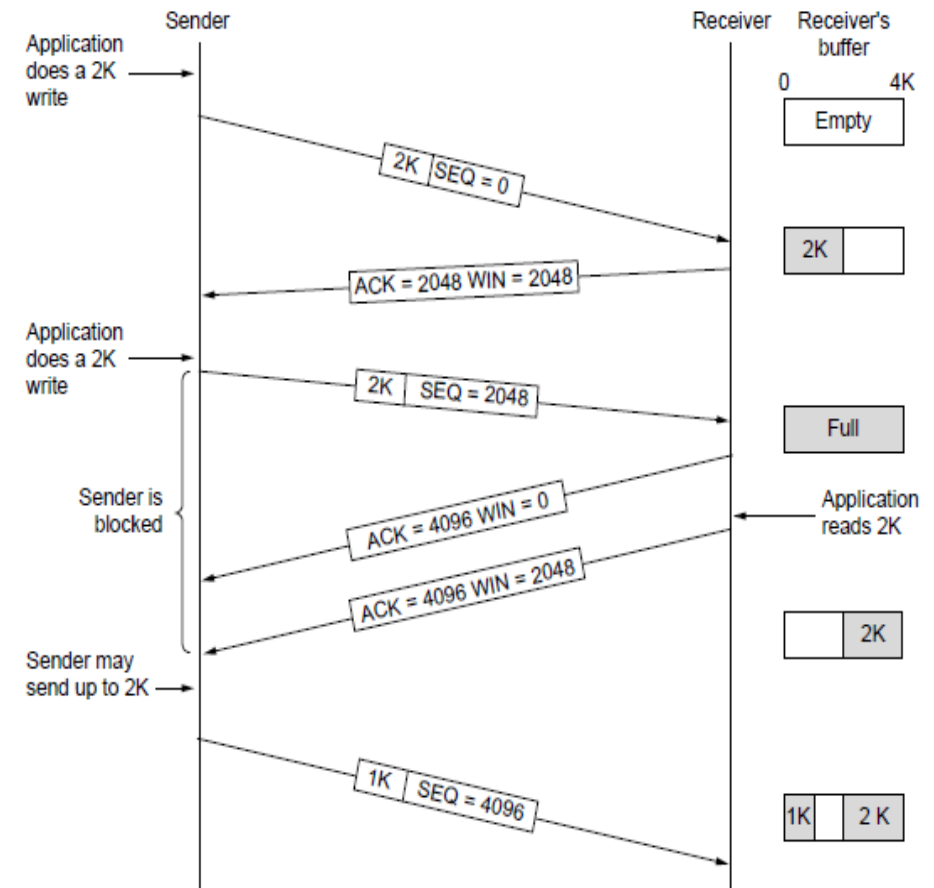
Se liberan 2 KB del búfer. El receptor anuncia **WIN = 2 KB**.

PASO 5

Búfer con 3 KB

El emisor usa su crédito y envía 1 KB más. Ahora el búfer tiene 3 KB ocupados y 1 KB libre → **WIN = 1 KB**.

DIAGRAMA PARCIAL — PASOS 4-5



Resolución completa en la siguiente slide →

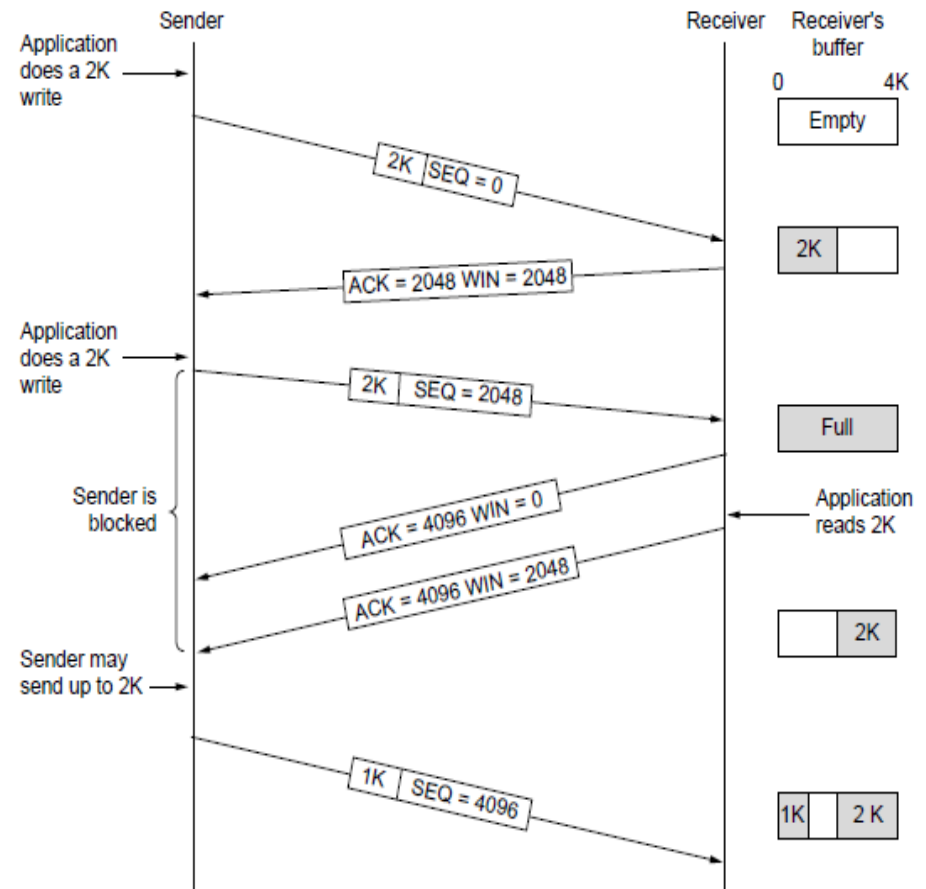
Diagrama de secuencia emisor ↔ receptor

CÓMO LEER EL DIAGRAMA

- ▶ **Flechas →** segmentos emisor→receptor (seq, tamaño).
- ▶ **Flechas ←** ACK + tamaño de ventana.
- ▶ **Fondo sombreado** estado del búfer circular.

OBSERVACIONES CLAVE

- Con búfer lleno, el receptor anuncia **WIN = 0** y el emisor se detiene.
- Cuando la app lee 2 KB, el receptor anuncia **WIN = 2 KB** y el emisor puede reanudar.



→ El búfer circular de 4 KB dicta el ritmo del emisor; ACK + WIN son el límite efectivo.

Pérdida de segmentos · solución NAK

Problema

¿Cómo recuperar segmentos perdidos sin retransmitirlos todos?

SOLUCIÓN 1 · NAK (Negative Acknowledgement)

Cuándo se envía un NAK

El receptor detecta una brecha entre el número de secuencia esperado y el recibido.

Qué contiene el NAK

Solicita explícitamente los segmentos faltantes mediante un campo de opciones.

Después de rellenar las brechas

Una vez recibidos los segmentos faltantes, el receptor **envía un ACK acumulativo** confirmando todos los datos que tiene en búfer.

EN TCP ESTÁNDAR NO EXISTE NAK EXPLÍCITO

TCP no define un mensaje NAK. En su lugar, el receptor reenvía el **mismo ACK acumulativo** (ACK duplicado) por cada segmento que llega fuera de orden. Al recibir **3 ACKs duplicados**, el emisor dispara **Fast Retransmit** sin esperar el timeout (RFC 5681).

Pérdida de segmentos · ACKs selectivos (SACK)

Solución 2 · SACK

El receptor le dice al emisor exactamente qué rangos de bytes ha recibido, así este reenvía sólo lo que falta.

DOS OPCIONES DENTRO DE LOS SEGMENTOS TCP

SACK-Permitted option

TCP · Kind = 4 · Length = 2 bytes

Cuándo

En el campo **Options** del segmento SYN y SYN-ACK del three-way handshake.

Qué indica

Que el extremo soporta SACK. Si ambos la envían, la conexión usará SACK (negociación RFC 2018).

SACK option

TCP · Kind = 5 · hasta 4 bloques por opción

Cuándo

El receptor la **adjunta dentro del campo Options** de los ACKs cuando detecta huecos en la secuencia — viaja en el mismo segmento.

Qué contiene

Pares (**Left Edge, Right Edge**) de 32 bits cada uno — los bloques recibidos fuera de orden por encima del ACK acumulativo.

→ Con SACK el emisor sabe exactamente qué falta: **retransmisión más precisa y eficiente** que con ACK acumulativo simple. **Pero estas son opciones, no obligaciones de implementación.**

Ej.: ACK=1001 + SACK=[2001–3001, 4001–5001] → faltan los bytes 1001–2000 y 3001–4000.

Ventana cero: dos excepciones permitidas

Recordatorio: con $rwnd = 0$, el emisor no puede enviar datos normales.

PERO HAY DOS EXCEPCIONES

EXCEPCIÓN 1

Datos urgentes

Pueden enviarse datos marcados como urgentes (p.ej. para que el usuario aborte un proceso en la máquina remota).

EXCEPCIÓN 2

Sonda de 1 byte

El emisor puede enviar un segmento de 1 byte para forzar al receptor a re-anunciar el siguiente byte esperado y el tamaño de ventana.

¿Por qué existe la sonda?

Si el aviso de ventana del receptor se pierde, el emisor podría esperar indefinidamente. La sonda de 1 byte **evita el bloqueo irreversible** forzando una nueva actualización de $rwnd$.

Problema: la ventana de 64 KB es pequeña

Problema

El campo **Window Size** de la cabecera TCP es de **16 bits**, por lo que la ventana anunciada no puede superar $2^{16}-1 = 65\,535 \text{ bytes}$ (**~64 KB**). En enlaces con **mucho ancho de banda y mucho retardo**, ese máximo no alcanza para llenar el pipe.

EJEMPLO · ENLACE T3

Capacidad
44.7 Mbps

RTT
50 ms

BDP mín.
~275 KB

Ventana máx.
64 KB

Consecuencia

Con ventana de 64 KB, el emisor estaría **inactivo $\frac{3}{4}$ del tiempo** esperando ACKs, en lugar de aprovechar el enlace.

→ Cambiar el campo de 16 bits rompería compatibilidad. La solución estándar es **escalar** ese valor mediante una opción TCP (siguiente slide).

Solución: opción Window Scale

Solución

Ambos extremos acuerdan un **factor de escala** aplicado al campo de ventana: tamaño efectivo = $\text{rwnd} \times 2^k$.

CÓMO FUNCIONA

Negociación

Durante el SYN del handshake, cada extremo indica cuántos bits desplazar (hasta 14).

Tamaño máximo resultante

$2^{16} \times 2^{14} = 2^{30}$ bytes $\approx$ 1 GB de ventana.

Hoy en día

Window Scale es **estándar en las implementaciones TCP actuales**; sin él, las conexiones en redes rápidas no alcanzarían el throughput disponible.

PARTE FINAL

Ejercicios de repaso

Tres problemas sobre control de flujo TCP — con solución paso a paso

1. Concepto · 2. Cálculo de rwnd · 3. Traza temporal

¿Qué ocurre cuando el receptor anuncia WIN = 0?

ENUNCIADO

Durante una conexión TCP, el emisor recibe un ACK con WIN = 0.

Responde:

- (a) ¿Qué significa para el emisor?
- (b) ¿Qué podría ocurrir si se quedase a la espera de un nuevo anuncio de ventana?
- (c) ¿Cómo resuelve TCP esa situación?

SOLUCIÓN

- (a) El búfer de recepción está **lleno**. El emisor debe **detener el envío** de datos nuevos.
- (b) El ACK que libera el búfer podría **perderse**. El emisor quedaría **bloqueado indefinidamente** (interbloqueo).
- (c) El emisor arranca un **temporizador de persistencia** y envía periódicamente **segmentos de sondeo** (1 byte) que fuerzan al receptor a contestar con el WIN actualizado.

Así se garantiza que el emisor se entere en cuanto el receptor libere espacio, aun si los ACK se pierden.

Calcular rwnd y bytes que el emisor puede enviar

ENUNCIADO

El receptor tiene un **RcvBuffer = 16 KB**.

- Ha recibido **12 KB** de datos.
- La aplicación ha leído **5 KB**.
- El emisor ya tiene **2 KB** en vuelo no confirmados.

Calcula:

- (a) el valor de **rwnd** que anuncia el receptor
- (b) los **bytes adicionales** que el emisor puede enviar

SOLUCIÓN

Fórmula: $\text{rwnd} = \text{RcvBuffer} - (\text{datos recibidos} - \text{datos leídos})$

Paso 1 — datos en búfer:

$12 \text{ KB} - 5 \text{ KB} = 7 \text{ KB}$ ocupando búfer

Paso 2 — (a) rwnd anunciada:

$16 - 7 = \text{rwnd} = 9 \text{ KB}$

Paso 3 — (b) nuevos bytes enviables:

$\text{rwnd} - \text{bytes en vuelo}$

$9 - 2 = 7 \text{ KB}$

Traza temporal: ¿qué WIN se anuncia tras cada evento?

Receptor con RcvBuffer = 10 KB. Estado inicial: búfer vacío, aplicación no ha leído nada. Completa WIN anunciado en cada evento.

Evento	Recibido (acum.)	Leído (acum.)	WIN anunciado
① Llegan 3 KB	3 KB	0 KB	7 KB
② App lee 2 KB	3 KB	2 KB	9 KB
③ Llegan 8 KB	11 KB	2 KB	1 KB
④ Llegan 1 KB más	12 KB	2 KB	0 KB ⚠
⑤ App lee 6 KB	12 KB	8 KB	6 KB

CLAVE PARA RESOLVERLO

$$\text{WIN} = \text{RcvBuffer} - (\text{Recibido} - \text{Leído})$$

En el evento ④ el búfer está lleno y se anuncia **WIN = 0**: el emisor debe detenerse y el temporizador de persistencia entra en juego. Cuando en ⑤ la aplicación lee 6 KB, se libera espacio y el siguiente ACK reabre la ventana a **6 KB**.